The Orange T-Adaptive Segment Routing Problem: A Maximum-Link-Utilization Approach

Nguyen Trong Tuan
HUST
Tuan.NT261067M@sis.hust.edu.vn

Vuong Dinh Minh
HUST
Minh.VD261073M@sis.hust.edu.vn

Abstract

This report studies the primary optimization stage of the EURO/ROADEF 2026 T-Adaptive Segment Routing problem. For each traffic demand and period, the model selects a bounded segment path while accounting for shortest-path traffic splitting, scheduled link withdrawals, and a limit on inter-period reconfiguration. The objective minimizes maximum link utilization (MLU), a standard traffic-engineering criterion that controls the network bottleneck and preserves residual capacity. We formulate the problem as a mixed-integer program and combine a compact formulation with a path-based column-generation strategy to address the growth in demand–segment variables. The method is evaluated on all twenty Set A instances using ten seeded runs, with limits of 1800 s for period 0 and 3600 s for the joint two-period model. All ten runs are certified MLU-optimal for 12 of 20 period-0 instances and 13 of 20 two-period instances; the remaining cases retain explicit lower bounds and optimality gaps. MLU is the first component of the challenge's sorted lexicographic load objective, so these certificates establish scalar MLU optimality but do not optimize or certify the subsequent load ranks.

1 Introduction

Context. Orange operates large national and international IP/MPLS backbones containing thousands of routers and links and carrying many thousands of origin–destination traffic demands. The continued growth of traffic, the diversity of services, and the development of Software-Defined Networking and Network Virtualization make the efficient use of this infrastructure increasingly important [6]. Traffic Engineering (TE) addresses this operational problem by selecting routing parameters and paths so that the available capacity is used efficiently while an acceptable Quality of Service (QoS) is maintained. In particular, routing should avoid local congestion: an overloaded link can degrade every service whose traffic crosses it even when unused capacity remains elsewhere in the network.

Motivation. Capacity efficiency alone is not sufficient. Operator networks must also remain available when equipment is taken out of service, whether because of an unforeseen failure or a planned maintenance intervention. The subject recalls the standard requirement that a network remain functional after any single link or node failure, while current security and climate-related risks further motivate resilience to wider disruptions [6]. Traffic volumes and equipment availability also vary with time. A routing plan should therefore anticipate forecast traffic matrices and scheduled interventions rather than optimize only one static nominal state. At the same time, excessive reconfiguration is undesirable: adding or removing waypoints may require operational actions, can incur a cost, and can temporarily affect QoS. The problem must consequently balance congestion reduction, resilience, and routing stability.

Traditional IP routing forwards packets along shortest paths induced by administrative link weights. This mechanism is simple and scalable, but the set of weights may not provide enough flexibility to distribute traffic well under several traffic and availability scenarios. Segment Routing (SR) [3] introduces a limited sequence of node segments, or waypoints, into the packet header. Between two consecutive waypoints, forwarding still follows the existing shortest-path mechanism. SR can therefore steer a demand through a non-shortest end-to-end route without requiring a separate forwarding state for every demand at every router, providing additional TE flexibility with limited implementation overhead.

Problem description. The EURO/ROADEF 2026 challenge defines the T-Adaptive Segment Routing problem (T-ASR) [6]. Its input is a capacitated directed network with administrative weights, a set of demands and time-dependent traffic volumes, a planning horizon $T = \{0, \dots, h-1\}$, scheduled link interventions, a reconfiguration budget, and a maximum number of segments. For every demand and every time period, the decision is a segment path from the demand source to its destination. The selected paths must satisfy flow conservation and the segment limit; the induced traffic must respect the available period graph; and consecutive routing schemes must remain within the reconfiguration budget. The resulting link load is the carried flow divided by link capacity and may exceed one when the network is congested.

The full challenge objective sorts all link loads across all periods in non-increasing order and minimizes this vector lexicographically: first the largest load, then the second largest, and so forth [6]. The implementation studied here solves the scalar primary stage, the minimization of maximum link utilization (MLU). This is the first component of the lexicographic objective and a standard TE criterion; its scientific rationale and precise scope are detailed in Section 2.3.

Scope of this report. Following the course instructions, the model is first evaluated on period 0 alone, which is the nominal network without the reconfiguration-budget constraint. It is then extended to two periods: period 1 applies the scheduled link withdrawals, uses its own traffic volumes, and activates the budget constraint between the two routing schemes. All twenty Set A instances are tested with a time limit of 1800 s for period 0 and 3600 s for the two-period model. Each reported result is an average over ten runs with seeds 2026–2035. The remainder of the report presents the data construction, mathematical formulation, solution method, experimental protocol, numerical results, and limitations of the scalar MLU study.

2 Mathematical model

2.1 Data

Let V be the set of routers and A the set of directed links in the nominal network. Every link $a = (u, v) \in A$ has a strictly positive administrative metric ω_a and a capacity $c_a > 0$. Let D be the set of traffic demands. A demand $d \in D$ is specified by its source $s_d \in V$, destination $\tau_d \in V$, and nonnegative volume v_{dt} in period t. The experiments use either $T = \{0\}$ or $T = \{0, 1\}$. For each period, $q_t \subseteq A$ denotes the links removed by the intervention and

$$A_t = A \setminus q_t, \qquad G_t = (V, A_t).$$

In the supplied instances $q_0 = \varnothing$. The scenario also provides the maximum number of segments $M = \mathrm{maxSeg} = 6$ and, for the transition from period 0 to period 1, a reconfiguration budget κ.

These parameters are read from the three files required by the project specification: the network file supplies (V, A, ω, c), the traffic-matrix file supplies (D, s, τ, v), and the scenario file supplies (q, M, κ). The JSON representation is therefore only the input format; the optimization model below is defined in terms of the mathematical sets and parameters above. Node and link identifiers are mapped explicitly rather than assumed to be consecutive.

Only a reachable ordered pair (i, j) with $i \neq j$ can be used as a segment in period t; let S_t denote the set of all such pairs. Between the endpoints of a segment, traffic is forwarded on shortest paths in G_t. Let ℓ^t_{ij} be the shortest-path distance from i to j and let σ^t_{ij} be the number of shortest directed paths. For an available link $a = (u, v) \in A_t$, the split coefficient r_{ijat} is the fraction of the traffic of segment (i, j) carried by a. The course note [1] gives

$$r_{ijat} = \begin{cases} \dfrac{\sigma^t_{iu}\,\sigma^t_{vj}}{\sigma^t_{ij}}, & \text{if } \ell^t_{iu} + \omega_a + \ell^t_{vj} = \ell^t_{ij}, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

All distances and path counts, and hence all r_{ijat}, are recomputed after an intervention because deleting a link can change both reachability and the shortest-path DAG. The implementation runs Dijkstra's algorithm from every router and stores path counts in logarithmic form to avoid numerical overflow.

Equation (1) assigns equal probability to complete shortest paths and is used throughout the experiments. It is not identical to a router-local ECMP recursion, which can assign different probabilities to complete shortest paths on an asymmetric DAG. Consequently, the numerical loads in this report are internally consistent with the supplied course note but may differ from an evaluator that applies router-local ECMP.

2.2 Decision variables

For every $d \in D$, $t \in T$, and $(i, j) \in S_t$, define

$$x_{dtij} = \begin{cases} 1, & \text{if the segment path of demand } d \text{ contains segment } (i, j) \text{ in period } t, \\ 0, & \text{otherwise.} \end{cases}$$

Thus, the nonzero x variables describe the ordered waypoint sequence selected for each demand and period; the physical links used between two consecutive waypoints are induced by the split coefficients rather than selected directly.

The continuous variable $L \geq 0$ is an upper bound on every available-link utilization in every modeled period. No separate load variable is required because each load is a linear expression in x. For the two-period model, an auxiliary continuous variable $y_{dij} \geq 0$ represents the absolute change of segment (i, j) for demand d between periods.

2.3 Objective

For a link $a \in A_t$, define its carried flow and utilization as

$$F_{at} = \sum_{d \in D} \sum_{(i,j) \in S_t} r_{ijat} v_{dt} x_{dtij}, \qquad \lambda_{at} = \frac{F_{at}}{c_a}. \tag{2}$$

The implemented objective is the scalar maximum link utilization (MLU):

$$\min L, \qquad L = \max_{t \in T,\ a \in A_t} \lambda_{at}. \tag{3}$$

This classical intra-domain traffic-engineering criterion suppresses bottlenecks, distributes traffic over the available capacity, and preserves residual capacity for growth, bursts, and failures [4, 5]. Because queuing delay rises nonlinearly as utilization approaches capacity, controlling the peak is more protective of worst-case performance than minimizing average load alone [4].

The challenge objective [6] is stronger: after minimizing the largest element of the load vector, it minimizes the second largest, then the third largest, and so on. Objective (3) is exactly the first stage of that lexicographic objective, but it does not break ties between solutions having the same MLU. Accordingly, every lower bound, gap, and "MLU-optimal" label in this report refers only to the scalar problem (3).

2.4 Constraints

Flow conservation. For each demand and period, the selected segments must form a unit flow from its source to its destination:

$$\sum_{j:(i,j) \in S_t} x_{dtij} - \sum_{j:(j,i) \in S_t} x_{dtji} = \begin{cases} 1, & i = s_d, \\ -1, & i = \tau_d, \\ 0, & \text{otherwise}, \end{cases} \qquad \forall d \in D,\ t \in T,\ i \in V. \tag{4}$$

The formulation also adds the valid degree inequalities

$$\sum_{j:(i,j) \in S_t} x_{dtij} \le \mathbf{1}\{i \ne \tau_d\}, \qquad \sum_{j:(j,i) \in S_t} x_{dtji} \le \mathbf{1}\{i \ne s_d\}, \quad \forall d, t, i. \tag{5}$$

They forbid branching, forbid a segment entering the source or leaving the destination, and make the source–destination component a simple path. A disconnected directed cycle is still formally possible, but deleting it preserves conservation and cannot increase L, since every traffic coefficient is nonnegative.

Segment limit. The complete segment path, including its first and last segments, is limited by

$$\sum_{(i,j) \in S_t} x_{dtij} \le M, \qquad \forall d \in D,\ t \in T. \tag{6}$$

This is the bound on the segment list carried by the packet, not a bound on the number of physical links traversed inside the shortest-path forwarding graphs.

Link utilization. Substituting (2) into $\lambda_{at} \le L$ gives the linear constraints

$$\sum_{d \in D} \sum_{(i,j) \in S_t} r_{ijat} v_{dt} x_{dtij} \le c_a L, \qquad \forall t \in T,\ a \in A_t. \tag{7}$$

The left-hand side sums the fractions of all selected segments that traverse link a. Dividing by c_a shows directly that (7) bounds every link utilization by L; because L is minimized, at least one such inequality is tight at an optimum. Loads above one remain allowed and indicate congestion.

Reconfiguration budget. Let $S = S_0 \cup S_1$ and define $\bar{x}_{dtij} = x_{dtij}$ when $(i,j) \in S_t$, and $\bar{x}_{dtij} = 0$ otherwise. For each demand and possible segment, the absolute difference is linearized as

$$y_{dij} \ge \bar{x}_{d,1,ij} - \bar{x}_{d,0,ij}, \quad y_{dij} \ge \bar{x}_{d,0,ij} - \bar{x}_{d,1,ij}, \quad \forall d \in D,\ (i,j) \in S. \tag{8}$$

The total number of segment-usage changes is bounded by

$$\sum_{d \in D} \sum_{(i,j) \in S} y_{dij} \le \kappa. \tag{9}$$

The two inequalities force $y_{dij} \ge |\bar{x}_{d,1,ij} - \bar{x}_{d,0,ij}|$; any unnecessary excess only consumes budget and can be reduced without affecting another constraint. Hence (9) bounds the symmetric difference between the two segment sets exactly as required by the project definition. It is omitted in the period-0 experiment and activated only in the joint two-period model.

3 Solution method

3.1 Formulation selection

The direct implementation of Section 2 is a mixed-integer program with one binary segment variable for every demand, period, and reachable ordered pair of routers. This compact representation is exact for the scalar MLU model and exposes flow conservation, segment limits, link loads, and, in the two-period case, reconfiguration decisions directly to the solver. Its principal limitation is the $O(|D||T||V|^2)$ number of routing variables. The implementation therefore estimates this number as $|D||T||V|(|V|-1)$ and uses the compact formulation when the estimate is at most $100\,000$; larger instances are solved by the path-based method described below.

3.2 Single-period column generation

For a period t, let $\mathcal{P}_{dt}$ be the set of admissible segment paths for demand d, and let R_{pat} denote the fraction of path p carried by physical link a. A master variable z_{dtp} selects one complete path, so flow conservation and the segment limit are enforced by the definition of a column. For a restricted subset $\widehat{\mathcal{P}}_{dt} \subseteq \mathcal{P}_{dt}$, the LP master is

$$
\begin{aligned}
\min \quad & L, \\
\text{s.t.} \quad & \sum_{p \in \widehat{\mathcal{P}}_{dt}} z_{dtp} = 1, && \forall d, && (10)\\
& \sum_{d \in D} \sum_{p \in \widehat{\mathcal{P}}_{dt}} v_{dt} R_{pat} z_{dtp} \le c_a L, && \forall a \in A_t, && (11)\\
& z_{dtp} \ge 0.
\end{aligned}
$$

The initial pool contains the direct source–destination segment for every demand. After solving the restricted master, pricing uses the dual values of (10)–(11) to find a minimum-reduced-cost path with at most M segments. This bounded shortest-path problem is solved exactly by dynamic programming on M layers of the complete segment graph.

Let $\bar{c}^*_{dt}$ be the minimum reduced cost returned for demand d. At every pricing round,

$$\mathrm{LB}_t = z_{\mathrm{RMP}} + \sum_{d \in D} \min\{0, \bar{c}^*_{dt}\}$$

is a valid Lagrangian lower bound for the complete path master. The dynamic program minimizes over bounded walks, a superset of simple paths; therefore its value remains a valid lower estimate even if a minimum walk repeats a router. Cycles are removed before a feasible column is inserted. If no negative-reduced-cost path exists, the restricted LP value itself is the optimum of the complete LP relaxation. The generated variables are then made binary and the restricted integer master is solved to obtain a feasible routing and an upper bound.

3.3 Joint two-period solution

For the full experiment, column generation is first applied separately to periods 0 and 1. Each solve receives 40% of the total time limit and returns an initial path pool, an incumbent routing, and a valid single-period lower bound. Because the budget constraint only restricts the joint feasible set, $\max\{\mathrm{LB}_0, \mathrm{LB}_1\}$ is a valid initial lower bound for the full problem. The two pools are then enriched with a small number of feasible waypoint paths and with routes that are valid in both period graphs.

The pools are coupled in a joint master containing one path-selection constraint per demand and period, the link-load constraints of both periods, and the absolute-change linearization (8)–(9). An initial binary joint master, warm-started from a repaired budget-feasible pair of routings, provides a joint incumbent. The corresponding LP relaxation is then used for joint pricing. The reduced cost of a path includes the dual contribution of its period's link-load constraints, its demand-selection constraint, and the two absolute-change rows associated with every segment already represented in the joint master. For a previously unseen segment, assigning equal dual values to its two new change rows gives zero net reconfiguration penalty; this is a dual-feasible completion and hence preserves the validity of the lower bound.

Pricing is performed for every demand in both periods by the same bounded dynamic program. If $\bar{c}^*_{dt}$ is the resulting minimum reduced cost, the joint bound is

$$\mathrm{LB}_{\mathrm{joint}} = z_{\mathrm{joint\ LP}} + \sum_{t \in \{0,1\}} \sum_{d \in D} \min\{0, \bar{c}^*_{dt}\}.$$

Negative-reduced-cost paths are added and the joint LP is re-solved. The implementation certifies the complete joint LP when the correction term is zero. Practical safeguards limit joint pricing to 40 rounds and $25\,000$ columns and stop it after

six rounds without LP improvement; if a safeguard is reached first, the corrected Lagrangian value remains a valid, possibly weaker, bound. The reported global lower bound is

$$\mathrm{LB} = \max\{\mathrm{LB}_0, \mathrm{LB}_1, \mathrm{LB}_{\mathrm{joint}}\}.$$

Finally, the enlarged budget-coupled master is solved with binary path variables.

3.4 Use of the remaining time

Obtaining a valid relaxation bound does not imply that the best integer routing in the current column pool attains that bound. When a positive gap and usable time remain, the implementation therefore generates additional feasible columns using systematic one-waypoint paths, randomized two-waypoint paths, and bounded paths obtained from randomized segment costs. It warm-starts the enlarged binary master from the best validated routing and solves it again with a different random seed. For the period-0 experiment, this diversification is applied to the single-period master. For the full experiment, the independent period-level diversification is disabled after the initial pools have been constructed; a candidate is instead added to every period pool in which it is feasible, and the budget-coupled joint MIP is re-solved. The process stops when the gap closes, the time limit is reached, or no new path is found and the integer solve terminates immediately. These heuristic rounds can improve only the feasible upper bound; they do not modify the previously established relaxation lower bound.

3.5 Bounds and certification

After optimization, every incumbent is checked independently for correct terminals, reachability, segment count, repeated vertices, reconfiguration cost, and recomputed link loads. The checked maximum load is the reported upper bound UB. The compact solver or the column-generation procedure supplies the lower bound LB, and the reported gap is

$$\max\left\{0,\, 1 - \frac{\mathrm{LB}}{\mathrm{UB}}\right\}.$$

A run is labelled *MLU-optimal* when this relative gap is at most 10^{-6}, the numerical tolerance adopted by the implementation. This certification concerns the scalar objective (3); it does not certify the remaining ranks of the challenge's lexicographic objective.

4 Experimental protocol

The experiments use all twenty instances in Set A. Their sizes range from 20 to 400 vertices, 80 to 2000 directed links, and 20 to 6000 demands. Two experiments are performed for each instance. The first considers only period 0 and omits the reconfiguration budget, as required in Step 6 of the project. The second considers periods 0 and 1 together and includes the budget constraint, as required in Steps 7–9.

The period-0 time limit is 1800 seconds (30 minutes), while the two-period limit is 3600 seconds (one hour). Each experiment is repeated ten times using seeds 2026–2035. Every solver process uses one thread. The code is run with Julia 1.13.2, JuMP 1.32.1 [2], and HiGHS 1.26.0 on 64-bit Linux 6.8.0. The machine has two Intel Xeon Platinum 8558 processors, with 44 physical cores per socket and two hardware threads per core (88 physical cores and 176 logical CPUs in total), and 1.7 TiB of RAM. Although the machine has many cores, the experiments are executed sequentially: only one instance and one seed are active at a time, and HiGHS uses one thread. Thus, no two experimental jobs compete for CPU or memory. The operating-system background load was not logged, so no numerical load average is claimed.

For every run, the output file records the instance size, checked MLU, best valid lower bound, relative gap, solver time, number of branch-and-bound nodes, solution status, and solution method. In accordance with the project instructions, *solver time* includes only elapsed time inside calls to `optimize!`. When an algorithm makes several solver calls, these times and their branch-and-bound node counts are summed. Data loading, graph construction, coefficient computation, pricing, validation, and file writing are excluded. A run may stop before its limit when the lower bound certifies the incumbent. Small overruns are possible because a solver call is timed when it returns.

The original experiment files do not contain end-to-end wall-clock time. Consequently, the tables report the required solver time but do not present it as the total running time of the algorithm. In particular, a small solver time for a column-generation run does not include shortest-path preprocessing, pricing, or column construction. Measuring total wall-clock time would require rerunning the experiments with an outer timer.

Tables 1 and 2 report the best MLU, mean MLU, sample standard deviation of MLU, mean solver time, mean gap, mean node count, and number of certified runs out of ten. The gap of one run is

$$\mathrm{gap} = \max\left\{0,\, 1 - \frac{\mathrm{LB}}{\mathrm{UB}}\right\},$$

where UB is the checked MLU and LB is a valid lower bound. The *Cert.* column makes mixed outcomes explicit; for example, $8/10$ means that eight runs certify the scalar MLU optimum and two do not. An infeasible run would be reported separately, but every run in this study produced a feasible incumbent.

5 Results

5.1 Period-0 results

Table 1 summarizes the experiment that uses only period 0. All ten runs produced a feasible solution.

Table 1: Period 0 results over ten runs (limit: 1800 s). SD is the sample standard deviation of MLU; solver time, gap, and nodes are means.

Inst.	$\|V\|$	$\|A\|$	$\|D\|$	Best	Mean	SD	Solver (s)	Gap	Nodes	Cert.
A01	20	80	40	0.9294	0.9294	0	1.05	0	1.0	10/10
A02	30	150	45	0.5486	0.5486	0	2.79	0	1.0	10/10
A03	50	250	20	0.9435	0.9435	0	39.4	0	1.0	10/10
A04	50	250	200	0.4591	0.4591	0	1744.0	1.12×10^{-4}	105.7	0/10
A05	100	396	100	0.1155	0.1155	0	0.03	0	0.0	10/10
A06	100	500	500	0.0986	0.0986	0	352.3	0	795.0	10/10
A07	100	500	800	0.9080	0.9080	0	0.03	0	0.0	10/10
A08	150	654	200	0.3189	0.3189	0	2.69	0	2.1	10/10
A09	150	750	200	0.5054	0.5054	0	0.04	0	1.0	10/10
A10	150	966	1000	0.0761	0.0761	0	1739.3	0.1020	272.6	0/10
A11	200	1000	400	0.6555	0.6555	0	1741.0	0.0646	1.3	0/10
A12	200	898	400	0.6401	0.6401	0	1299.3	0.0955	9.9	0/10
A13	200	1000	2000	0.0389	0.0389	0	1741.5	9.99×10^{-4}	2690.6	0/10
A14	250	1108	600	0.4240	0.4240	0	1745.2	1.43×10^{-4}	119.2	0/10
A15	250	1250	600	0.8252	0.8252	0	0.04	0	0.0	10/10
A16	250	1452	4800	0.0389	0.0393	2.8×10^{-4}	1787.0	0.0172	12398.5	0/10
A17	300	1270	2000	0.4242	0.4242	0	0.05	0	0.0	10/10
A18	300	1500	2000	0.6334	0.6334	0	0.04	0	0.0	10/10
A19	300	1998	6000	0.0475	0.0483	4.5×10^{-4}	1774.2	0.0296	8516.1	0/10
A20	400	2000	6000	0.9913	0.9913	0	0.09	0	0.0	10/10

Twelve instances are certified MLU-optimal in all ten runs. Some large instances require very little solver time. For example, setA-20 has 400 vertices and 6000 demands, but its calls to `optimize!` take only 0.09 seconds on average because the lower bound already matches the feasible incumbent. This is not an end-to-end runtime: graph construction, pricing, and column generation are excluded.

The remaining eight instances are not certified within the available time. However, the final gaps for setA-04, setA-13, and setA-14 are below 10^{-3}, so their reported solutions are close to the corresponding lower bounds. In contrast, setA-10, setA-11, and setA-12 retain mean gaps of 10.20%, 6.46%, and 9.55%, respectively. SetA-16 and setA-19 also show some variation in their feasible objective values across the ten seeds; the best values found are 0.0389 and 0.0475, compared with the displayed means of 0.0393 and 0.0483.

5.2 Two-period results

The two-period model is certified MLU-optimal in all ten runs for thirteen instances. SetA-03 has mixed results: eight runs prove an optimum of 0.9435, while two runs reach 0.9444 before the time limit. The other six instances without a complete certificate are setA-04, setA-10, setA-13, setA-14, setA-16, and setA-19. Among them, setA-04 remains very close to its lower bound, with a mean gap of only 1.61×10^{-4}. SetA-16 and setA-19 are much harder for the implemented method, ending with mean gaps of 66.82% and 55.86%. These large gaps indicate weak certificates; they do not by themselves show that the feasible solutions are poor.

5.3 Comparison of the two experiments

Figure 1 makes the effect of adding period 1 easier to see. For setA-01, setA-06, setA-07, setA-08, setA-17, and setA-20, the certified two-period optimum is equal to the period-0 optimum. In these instances, the reconfiguration budget still

Table 2: Joint two-period results over ten runs with the reconfiguration budget (limit: 3600 s). SD is the sample standard deviation of MLU; solver time, gap, and nodes are means.

Inst.	$\|V\|$	$\|A\|$	$\|D\|$	Best	Mean	SD	Solver (s)	Gap	Nodes	Cert.
A01	20	80	40	0.9294	0.9294	0	14.6	0	1.0	10/10
A02	30	150	45	0.9031	0.9031	0	152.7	0	1.0	10/10
A03	50	250	20	0.9435	0.9437	3.7×10^{-4}	1430.5	0.0030	33.1	8/10
A04	50	250	200	0.5812	0.5812	1.1×10^{-7}	3592.7	1.61×10^{-4}	1503.4	0/10
A05	100	396	100	0.2050	0.2050	0	0.32	0	1.0	10/10
A06	100	500	500	0.0986	0.0986	0	168.0	0	540.8	10/10
A07	100	500	800	0.9080	0.9080	0	42.3	0	229.6	10/10
A08	150	654	200	0.3189	0.3189	0	1.26	0	3.0	10/10
A09	150	750	200	0.8497	0.8497	0	2.60	0	2.0	10/10
A10	150	966	1000	0.0717	0.0733	0.0013	3594.1	0.0678	4947.0	0/10
A11	200	1000	400	0.7858	0.7858	0	2.92	0	2.0	10/10
A12	200	898	400	0.8799	0.8799	0	1.82	0	2.0	10/10
A13	200	1000	2000	0.0415	0.0420	4.9×10^{-4}	3591.1	0.0288	7846.2	0/10
A14	250	1108	600	0.5176	0.5176	0	3608.9	0.0197	17922.4	0/10
A15	250	1250	600	0.8987	0.8987	0	5.61	0	1.0	10/10
A16	250	1452	4800	0.1025	0.1369	0.0245	3594.2	0.6682	32238.2	0/10
A17	300	1270	2000	0.4242	0.4242	0	50.5	0	1.0	10/10
A18	300	1500	2000	1.0000	1.0000	0	32.8	0	0.0	10/10
A19	300	1998	6000	0.0796	0.1096	0.0199	3542.7	0.5586	15254.0	0/10
A20	400	2000	6000	0.9913	0.9913	0	401.9	0	2.0	10/10

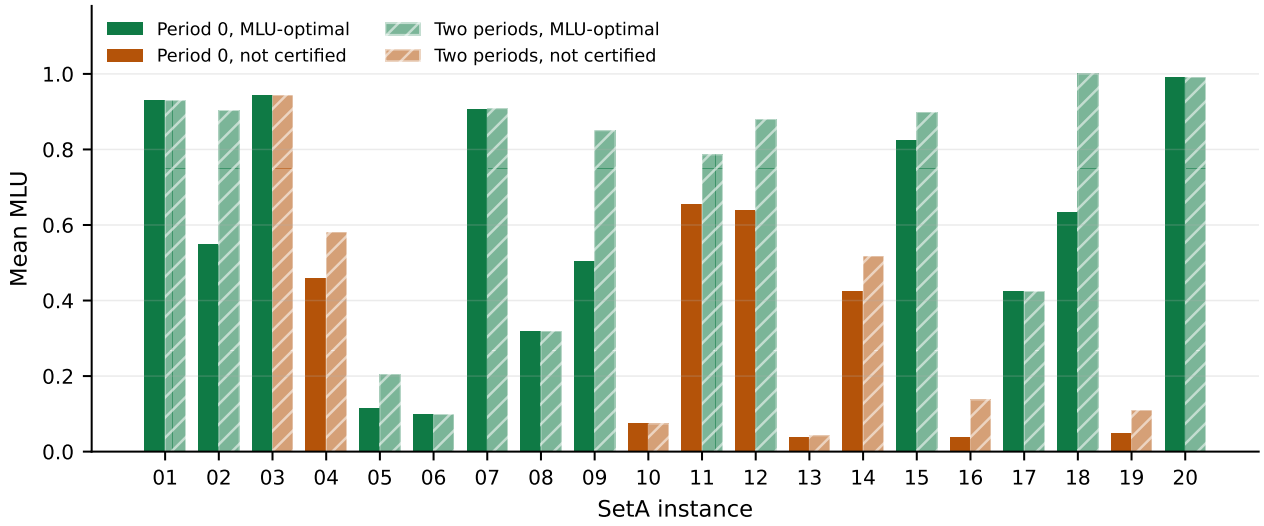


Figure 1: Mean MLU for period 0 (solid bars) and for the joint two-period model (hatched bars). Green indicates that all ten runs are certified MLU-optimal; amber indicates that at least one run is not certified.

allows routes whose peak utilization does not increase. For several other certified cases, the second period raises the MLU substantially: from 0.5486 to 0.9031 on setA-02, from 0.1155 to 0.2050 on setA-05, from 0.5054 to 0.8497 on setA-09, and from 0.6334 to 1.0000 on setA-18.

The two tables should be compared carefully when either row is not certified. For example, the mean feasible MLU of setA-10 is lower in the two-period experiment than in the period-0 experiment. This does not mean that adding a period improves the true optimum. Both rows are non-optimal search results, and the longer two-period run simply found a better feasible period-0-compatible routing. The same caution applies to comparisons involving setA-13, setA-16, and setA-19.

6 Discussion and limitations

The experiments show that model size is not a reliable predictor of the reported solver time. Some of the largest instances require very little time inside `optimize!`, whereas several smaller instances use most of the limit. The important factor is whether the relaxation produces a lower bound that is close to a good integer routing. When the two values meet, optimality can be certified without exploring a large branch-and-bound tree. When they remain far apart, a long search may still leave a nonzero gap.

The reconfiguration budget makes this issue more visible in the two-period experiment. Although the implementation combines valid single-period lower bounds with a joint relaxation bound, the resulting lower bound can still be weak for difficult instances. This is especially relevant for setA-16 and setA-19, whose mean gaps after one hour are 66.82% and 55.86%. Their best feasible MLU values over the ten runs are 0.1025 and 0.0796, respectively, both noticeably lower than their reported means. The variation between seeds suggests that additional path generation can still improve the incumbent, while the large gaps indicate that the current bounds are insufficient to assess how close those incumbents are to the true optimum.

The averages also hide differences between individual runs. SetA-03 is a clear example: eight runs prove the same two-period optimum, but two runs reach the time limit with a slightly worse solution. For this reason, the table marks the instance as not certified rather than treating its small mean gap as proof of optimality. Similarly, a zero branch-and-bound node count does not mean that no optimization was performed; the relaxation and column-generation stages may already have closed the gap at the root. Node counts should also be compared with care because they are summed over solver calls and come from two different formulations.

The timing results have a further limitation. They follow the course definition and measure only calls to `optimize!`; they are not end-to-end running times. This distinction matters most for column generation, where shortest-path preprocessing, pricing, and construction of new columns take place outside the solver calls. Since total wall-clock time was not stored in the result files, it cannot be reconstructed reliably afterwards. The present tables can therefore compare solver effort, but not the complete runtime of the two solution approaches.

Three limitations define the scope of the conclusions. First, the model minimizes only the maximum link utilization in (3). The full challenge objective is lexicographic: after minimizing the largest load, it also minimizes the second largest load, then the third largest, and so on [6]. Two solutions with the same MLU may therefore have different values under the official objective. Second, the coefficients r_{ijat} follow the formula given in the course note [1]. This formula assigns equal weight to complete shortest paths and may differ from router-local ECMP splitting on an asymmetric shortest-path graph. Third, the path-based integer model contains only the columns generated or added during the run. Its incumbents are feasible for the original model, and the reported lower bounds remain valid, but a non-certified row is not proof that the best possible routing has been found.

Future improvements could focus on strengthening the joint relaxation, improving the pricing strategy, and incorporating branch-and-price so that new paths remain available throughout the integer search. After the scalar MLU is certified, the same framework could also be extended to optimize the remaining levels of the lexicographic objective.

7 Conclusion

This project developed a complete workflow for the Orange T-Adaptive Segment Routing problem. The three JSON input files are converted into a period-specific network model, the shortest-path splitting coefficients are computed, and the resulting routing problem is solved as a mixed-integer linear program. A compact formulation is used for small instances, while a path-based formulation with column generation makes the larger instances tractable. Every reported routing is checked independently before its MLU is accepted.

The method was tested on all twenty Set A instances using the time limits specified by the course project. For period 0, twelve instances were certified MLU-optimal in all ten runs. For the joint two-period problem, thirteen instances were certified in all ten runs, while setA-03 was certified in eight of ten runs. Feasible solutions were found for every instance. The remaining large optimality gaps, particularly on setA-16 and setA-19, highlight the need for stronger lower bounds and improved feasible solutions in the current two-period approach.

The reported certificates apply to the scalar maximum-link-utilization objective (3), which is only the first level of the challenge's lexicographic objective. Future work could focus on improving lower-bound quality, refining the pricing strategy, and strengthening the integer search through branch-and-price techniques. Once the minimum MLU has been certified, the model could then optimize the remaining load levels in lexicographic order.

References

[1] Course note. Computing arc-use ratios on shortest paths, 2026.

[2] Iain Dunning, Joey Huchette, and Miles Lubin. JuMP: A modeling language for mathematical optimization. *SIAM Review*, 59(2):295–320, 2017.

[3] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir. Segment routing architecture. IETF RFC 8402, 2018.

[4] Bernard Fortz, Jennifer Rexford, and Mikkel Thorup. Traffic engineering with traditional IP routing protocols. *IEEE Communications Magazine*, 40 (10):118–124, 2002.

[5] Srikanth Kandula, Dina Katabi, Bruce Davie, and Anna Charny. Walking the tightrope: Responsive yet stable traffic engineering. In *Proceedings of ACM SIGCOMM*, pages 253–264, 2005.

[6] Orange Research. Challenge EURO/ROADEF 2026 problem description, 2026.